

Stawberries 3

MA615

2024 Sept 30

```
{r setup, include=FALSE} knitr::opts_chunk$set(echo = TRUE)
```

Version 6

We ditch the counties

Preparing data for analysis

Acquire, explore, clean & structure, EDA

Data cleaning and organization

“An introduction to data cleaning with R” by Edwin de Jonge and Mark van der Loo

“Problems, Methods, and Challenges in Comprehensive Data Cleansing” by Heiko Müller and Johann-Christoph Freytag

Strawberries

Questions

Where they are grown? By whom?

Are they really loaded with carcinogenic poisons?

Are they really good for your health? Bad for your health?

Are organic strawberries carriers of deadly diseases?

When I go to the market should I buy conventional or organic strawberries?

Do Strawberry farmers make money?

How do the strawberries I buy get to my market?

The data

The data set for this assignment has been selected from:

[USDA_NASS_strawb_2024SEP25 The data have been stored on NASS here:
USDA_NASS_strawb_2024SEP25

and has been stored on the blackboard as strawberries25_v3.csv.

read and explore the data

Set-up

```
``{r} #| label: load libraries and set options #| warning: false #| message: false #|  
echo: false
```

```
library(knitr)  
library(kableExtra) library(tidyverse)
```

Read the data and take a first look

```
``{r}  
#| label: read data - glimpse  
  
strawberry <- read_csv("strawberries25_v3.csv", col_names = TRUE, show_  
col_types = FALSE )  
  
## glimpse(strawberry)
```

Ditch the counties

```
``{r} #| label: ditch the counties  
  
unique(strawberry$Geo Level)  
  
strawberry <- strawberry |> filter(Geo Level=="NATIONAL" | Geo Level==  
"STATE")
```

I have 5359 rows and 21 columns.

remove columns with a single value in all rows

```
``{r}  
#|label: function def - drop 1-item columns  
  
drop_one_value_col <- function(df){  
  # browser()  
  df_id <- ensym(df)  
  msg = paste("Looking for single value columns in data frame: ",as.cha  
racter(df_id) )  
  print(msg)  
  ## takes whole dataframe  
  dropc <- NULL  
  val <- NULL  
  ## test each column for a single value  
  for(i in 1:dim(df)[2]){  
    if(dim(distinct(df[,i]))[1] == 1){
```

```

    dropc <- c(dropc, i)
    val <- c(val, df[1,i])
  }
}

if(is.null(dropc)){
  print("No columns dropped")
  return(df)}else{
  print("Columns dropped:")
  # print(colnames(df)[drop])
  print(unlist(val))
  df <- df[, -1*dropc]
  return(df)
}
}

```

use the function

```
strawberry <- strawberry |> drop_one_value_col()
```

Split the census data from the survey data. drop single value columns

Census data first

```
```{r} #| label: strawberries split census, survey
```

```
straw_cen <- strawberry |> filter(Program=="CENSUS")
```

```
straw_sur <- strawberry |> filter(Program=="SURVEY")
```

```
straw_cen <- straw_cen |> drop_one_value_col()
```

```
straw_sur <- straw_sur |> drop_one_value_col()
```

```
```{r}
```

```
#| label: straw_cen split cols
```

```
straw_cen <- straw_cen |>
  separate_wider_delim( cols = `Data Item`,
                        delim = " - ",
                        names = c("strawberries",
                                  "Category"),
                        too_many = "error",
                        too_few = "align_start"
                      )
```

```
```{r} #| label: isolate organic
```

```
unique(straw_cen$strawberries) # straw_cen$strawberries |>
str_which("STRAWBERRIES") |> length()
```

```
straw_cen$strawberries |> str_which("STRAWBERRIES,
ORGANIC") |> length() # straw_cen$strawberries |>
str_which("STRAWBERRIES, ORGANIC, FRESH MARKET") |>
length()
```

```
straw_cen$strawberries |> str_which("STRAWBERRIES,
ORGANIC, PROCESSING") |> length() # # ## count the cases #
straw_cen$strawberries |> str_which("ORGANIC") |> length()
```

```
straw_cen$strawberries |> str_which("FRESH MARKET") |>
length() # straw_cen$strawberries |> str_which("PROCESSING")
|> length()
```

```
straw_cen <- straw_cen |> separate_wider_delim(cols = strawberries, delim = ",",
names = c("strawberries", "ORGANIC", "organic_detail"),
```

```
 too_many = "error",
 too_few = "align_start"
)
```

```
straw_cen <- straw_cen |> drop_one_value_col()
```

### how many organic rows?

```
organic_cen <- straw_cen |> filter(ORGANIC == "ORGANIC")
```

```
sum(is.na(straw_cen$ORGANIC))
```

```
straw_cen <- straw_cen[(is.na(straw_cen$ORGANIC)),]
```

```
straw_cen <- straw_cen |> drop_one_value_col()
```

Note that straw\_cen has only one year: 2022

Current stats Census date has been isolated and split between Organic and Conventional strawberries

```
```{r}
#| label: explore straw_cen$Domain Category
```

```
straw_cen <- straw_cen |>
  separate_wider_delim( cols = `Domain Category`,
                        delim = " ",
                        names = c("COL1",
                                  "COL2"),
                        too_many = "merge",
                        too_few = "align_start"
                      )
```

```
straw_cen$COL2 <- str_replace(straw_cen$COL2, "WITH ", "")
```

```
straw_cen <- straw_cen |> rename(Measure = COL1, Bearing_type= COL2)
```

```
```{r} #| label: explore straw_cen$Domain & Domain Category
```

**remove AREA GROWN and parens**

**change NOT SPECIFIED TO TOTAL**

```
straw_cen <- straw_cen |> rename(size_bracket = Domain Category)
```

```
straw_cen$size_bracket <- str_replace(straw_cen$size_bracket, "NOT SPECIFIED",
"TOTAL")
```

```
straw_cen$size_bracket <- str_replace(straw_cen$size_bracket, "AREA GROWN:", "")
```

```
```{r}
#| label: explore organic_cen
```

```
organic_cen <- organic_cen |> drop_one_value_col()
```

Now examine the Survey data

```
```{r} #| label: Survey data
```

## Data Item

### unique(straw\_sur\$Data Item)

```
straw_sur1 <- straw_sur |> separate_wider_delim(cols = Data Item, delim = ",",
names = c("straw", "mkt", "measure", "other"), too_many = "merge", too_few =
"align_start")
```

```
straw_sur2 <- straw_sur1 |> separate_wider_delim(cols = "straw", delim = " - ",
names = c("straw", "more"), too_many = "merge", too_few = "align_start")
```

```
rm(straw_sur, straw_sur1)
```

Shift data into alignment

```
`{r}
#| label: Shift data on a row

function shift_loc
Moves adjacent data cells in a data.frame on a single row
Use this function to fix alignment problems after separating
columns containing multiple columns of data.

Of course the working assumption is that there is room in the
data frame for the data you're shifting.
##
The data cells that are empty after the data shift are NA.
##
Input paramaters
##
df -- data frame
col_name -- name of colume where the left-most data item is located
dat_name -- name of data item in the column
num_col -- the number of columns is the same as the number of
adjacent data to be moved.
num_shift -- the number of rows to move the data
##

shift_loc <- function(df, col_name, dat_name, num_col, num_shift){
 # browser()
 col_num = which(colnames(df) == col_name)
 row_num = which(df[,col_num] == dat_name) ## calcs a vector of rows

 for(k in 1:length(row_num)){
 d = rep(0,num_col) ## storage for items to be moved
 for(i in 1:num_col){
 d[i] = df[row_num[k], col_num + i - 1]
```

```

 }
 for(i in 1:num_col){
 ra = row_num[k]
 cb = col_num + i - 1
 df[ra, cb] <- NA
 }
 for(j in 1:num_col){
 rc = row_num[k]
 cd = col_num + j - 1 + num_shift
 df[rc, cd] = d[j]
 }
 }
}
sprintf("Rows adjusted:")
print("%d",row_num)
return(df)
}

```

```
straw_sur2 <- straw_sur2 |> shift_loc("more", "PRICE RECEIVED", 2, 1)
```

```
straw_sur2 <- straw_sur2 |> shift_loc("more", "ACRES HARVESTED", 1, 1)
```

```
straw_sur2 <- straw_sur2 |> shift_loc("more", "ACRES PLANTED", 1, 1)
```

```
straw_sur2 <- straw_sur2 |> shift_loc("more", "PRODUCTION", 2, 1)
```

```
straw_sur2 <- straw_sur2 |> shift_loc("more", "YIELD", 2, 1)
```

```
straw_sur2 <- straw_sur2 |> shift_loc("more", "APPLICATIONS", 3, 1)
```

```
straw_sur2 <- straw_sur2 |> shift_loc("more", "TREATED", 3, 1)
```

```
straw_sur2 <- straw_sur2 |> drop_one_value_col()
```

Examine Domain

```
``{r} #| label: use domain to make chem, fert, total data frames
```

```
unique(straw_sur2$Domain)
```

```
straw_sur2 <- straw_sur2 |> separate_wider_delim(cols = Domain, delim = ",", names
= c("col1", "col2"), too_many = "merge", too_few = "align_start")
```

```
unique(straw_sur2$col1)
```

```
survey_d_total <- straw_sur2 |> filter(col1 == "TOTAL")
```

```
survey_d_chem <- straw_sur2 |> filter(col1 == "CHEMICAL")
```

```
survey_d_fert <- straw_sur2 |> filter(col1 == "FERTILIZER")
```

```
now look at totals
```

```
```{r}
```

```
survey_d_total <- survey_d_total |> drop_one_value_col()
```

```
### align terms
```

```
survey_d_total <- survey_d_total |> shift_loc("measure", "MEASURED IN  
$ / CWT", 1, 1 )
```

```
survey_d_total <- survey_d_total |> shift_loc("measure", "MEASURED IN  
$", 1, 1 )
```

```
survey_d_total <- survey_d_total |> shift_loc("measure", "MEASURED IN C  
WT", 1, 1 )
```

```
survey_d_total <- survey_d_total |> shift_loc("measure", "MEASURED IN T  
ONS", 1, 1 )
```

```
survey_d_total <- survey_d_total |> shift_loc("measure", "MEASURED IN C  
WT / ACRE", 1, 1 )
```

```
survey_d_total <- survey_d_total |> shift_loc("measure", "MEASURED IN T  
ONS / ACRE", 1, 1 )
```

```
#### split the mkt column
```

```
survey_d_total <- survey_d_total |>  
  separate_wider_delim(cols = mkt,  
                        delim = " - ",  
                        names = c("col3",  
                                  "col4"),
```



```
too_many = "merge",  
too_few = "align_start")
```

there are two markets for Strawberries – Fresh Marketing and Processing

make a table for each

from the Survey Totals

we have reports for

Markets: Fresh and Processing Operations: Growing and Production

```
``{r}
```

```
survey_d_total <- survey_d_total |> group_by(Year) |> group_by(Period) |>  
group_by(Geo Level) |> group_by(State) |> group_by(col3)
```

```
``{r}
```

```
survey_d_total_ca <- survey_d_total |>  
filter(State == "CALIFORNIA")
```

```
ca_tab <- survey_d_total_ca |> group_by(Year, Period  
)
```

```
##
```

```
ca_tab_22 <- survey_d_total_ca |> filter(Year == 2022)
```

```
ca_tab_22 <- ca_tab_22 |> drop_one_value_col()
```

```
ca_tab_22 <- ca_tab_22 |>  
filter(Period == "YEAR")
```

```
ca_tab_22 <- ca_tab_22 |>  
filter(Value != "(D)")
```

```
ca_tab_22 <- ca_tab_22 |> drop_one_value_col()
```

```
``{r} #| label: chemicals 2022
```

```
ca_chem_22 <- survey_d_chem |>  
filter(Year == 2021) |> filter(State == "CALIFORNIA")
```

```
ca_chem_22 <- ca_chem_22 |> drop_one_value_col()
```

The codes I use here form a summary of the top 10 states with the highest strawberries production. This answer the first question, which asks where are the strawberries growing.

```
```{r}
```

```
Q1
```

```
strawberry <- read_csv("strawberries25_v3.csv", col_names = TRUE, show_col_types = FALSE)
```

```
Create a summary of strawberry production by State and County
```

```
state_county_summary <- strawberry %>%
 group_by(State, County) %>%
 summarise(Count = n()) %>%
 arrange(desc(Count))
```

```
Display the summary table
```

```
print(state_county_summary)
```

```
state_summary <- state_county_summary %>%
 group_by(State) %>%
 summarise(Count = sum(Count)) %>%
 arrange(desc(Count))
```

```
Plot the top 10 states by strawberry production count
```

```
top_states <- head(state_summary, 10)
```

```
ggplot(top_states, aes(x = reorder(State, Count), y = Count)) +
 geom_bar(stat = "identity", fill = "orange") +
 coord_flip() +
 labs(title = "Top 10 States by Strawberry Production",
 x = "State", y = "Production Count") +
 theme_minimal()
```

The code here provides a list of the chemical data mainly used on strawberries. We can change the number in top\_chemicals variable to see the name of the chemical and then check whether or not there are carcinogenic poisons.

```
```{r} # Q2
```

Filter rows that contain information about chemicals used

```
chemical_data <- strawberry %>% filter(grepl("CHEMICAL", Domain  
Category)) %>% select(State, County, Domain Category)
```

Display the chemical data

```
print(head(chemical_data))
```

Summarize the use of different chemicals

```
chemical_summary <- chemical_data %>% group_by(Domain Category) %>%  
summarise(Count = n()) %>% arrange(desc(Count))
```

Plot the most common chemicals used on strawberries

```
top_chemicals <- head(chemical_summary, 10)  
  
ggplot(top_chemicals, aes(x = reorder(Domain Category, Count), y = Count)) +  
geom_bar(stat = "identity", fill = "red") + coord_flip() + labs(title = "Top 10  
Chemicals Used on Strawberries", x = "Chemical Type", y = "Count of Usage") +  
theme_minimal()
```

Question 3 asks whether the strawberries are good for our health or not. I think the Domain.Category column in strawberries25 can determine whether or not it is health, because organic fruits do not use pesticides during the cultivation process. Based on the plot, most of strawberries are organic, which are health.

```
```{r}  
Q3
Filter organic and non-organic data
organic_data <- strawberry %>%
 filter(grepl("ORGANIC STATUS: \\(NOP USDA CERTIFIED\\)", `Domain Category`))

Count the number of organic vs. non-organic data entries
organic_count <- nrow(organic_data)
non_organic_count <- nrow(strawberry) - organic_count

Create a data frame for visualization
organic_summary <- data.frame(
 Type = c("Organic", "Non-Organic"),
 Count = c(organic_count, non_organic_count)
)
```

```
Plot organic vs. non-organic production
ggplot(organic_summary, aes(x = Type, y = Count, fill = Type)) +
 geom_bar(stat = "identity") +
 labs(title = "Comparison of Organic vs. Non-Organic Strawberry Production",
 x = "Production Type", y = "Count") +
 theme_minimal()
```

There may be some columns relate to both the organic and deadly diseases to answer question 4, but I can't figure it out. As for question 5, I think we should buy organic strawberries since all the status of the organic strawberries are NOP USDA CERTIFIED.

```
```{r} # Q4 library(dplyr)
```

Filter the data to only include rows related to organic strawberries

```
organic_strawberries <- strawberry %>% filter(grepl("ORGANIC STATUS: \ (NOP
USDA CERTIFIED\)", Domain Category))
```

Check for diseases or harmful chemicals

```
diseases_info <- organic_strawberries %>% filter(grepl("DISEASE|PATHOGEN",
tolower(Domain Category)))
```

View the results

```
print(diseases_info)
```

Q5

Filter the data to only include rows related to organic strawberries

```
organic_strawberries <- strawberry %>% filter(grepl("ORGANIC STATUS: \ (NOP
USDA CERTIFIED\)", Domain Category))
```

Print the resulting data

```
print(organic_strawberries)
```

Alternatively, you can view the data in a more interactive way using:

```
View(organic_strawberries)
```

Question 6 asks do strawberry farmers make money. We choose the Data Item column to observe. Based on the number of operations listed in the plot, we can say that strawberry farmers make money in some states of US.

```
```{r}
Q6
Filter data related to operations with strawberries
operation_data <- strawberry %>%
 filter(grepl("OPERATIONS", `Data Item`))

Summarize the number of operations
operation_summary <- operation_data %>%
 group_by(State) %>%
 summarise(Operations = n()) %>%
 arrange(desc(Operations))

Plot the number of operations by state
top_operations <- head(operation_summary, 10)

ggplot(top_operations, aes(x = reorder(State, Operations), y = Operations)) +
 geom_bar(stat = "identity", fill = "blue") +
 coord_flip() +
 labs(title = "Number of Strawberry Farming Operations by State",
 x = "State", y = "Number of Operations") +
 theme_minimal()
```

Now what I did is splitting the chemical data into use, name and code. The new data has three new columns with the chemical data so that these data are displayed clearly.

```
```{r} # Install and load dplyr if (!require("dplyr")) install.packages("dplyr")
library(dplyr)
```

Add new columns, and the new columns are all the chemical data in the Domain Category column.

```
strawberries <- strawberry %>% mutate( use = ifelse(grepl("^CHEMICAL,", Domain
Category), sub("^CHEMICAL, (.?):", "\\1", Domain Category), NA), name =
ifelse(grepl(":(.?) = ", Domain Category), sub("^CHEMICAL, .: \\.\\.?)$", "\\1",
'Domain Category'), NA), code = ifelse(grepl("= (\\d+)\\$", Domain
Category), sub(".*= (\\d+)\\$", "\\1", Domain Category), NA) )
```

Display the updated data frame

```
head(strawberries)
```

```
```{r}
Load necessary libraries
library(tidyverse)

Load the data (replace 'strawberries.csv' with your file path)
library(readr)
strawberries <- read_csv("C:/Users/Dgy49137/Desktop/strawberries.csv")
View(strawberries)

Convert "Value" and "CV (%)" to numeric, forcing non-numeric values to
NA
strawberries$Value_numeric <- as.numeric(strawberries$Value)
strawberries$CV_numeric <- as.numeric(strawberries$`CV (%)`)

Prepare the data: remove rows where both Value and CV are NA
df_clean <- strawberries %>%
 filter(!is.na(Value_numeric) | !is.na(CV_numeric))

Predict missing values for 'Value_numeric' and 'CV_numeric'
missing_values <- strawberries %>%
 filter(is.na(Value_numeric) | is.na(CV_numeric))

Check how many rows need prediction
num_missing_value <- sum(is.na(strawberries$Value_numeric))
num_missing_cv <- sum(is.na(strawberries$CV_numeric))

Predict Value_numeric for missing rows
predicted_value <- predict(model_value, newdata = missing_values)

Ensure the length of the predictions matches the number of missing rows
if (length(predicted_value) == num_missing_value) {
```

```

 strawberries$Value_numeric[is.na(strawberries$Value_numeric)] <- predicted_value
 } else {
 warning("The number of predicted 'Value_numeric' does not match the missing entries.")
 }

Predict CV_numeric for missing rows
predicted_cv <- predict(model_cv, newdata = missing_values)

Ensure the length of the predictions matches the number of missing rows
if (length(predicted_cv) == num_missing_cv) {
 strawberries$CV_numeric[is.na(strawberries$CV_numeric)] <- predicted_cv
} else {
 warning("The number of predicted 'CV_numeric' does not match the missing entries.")
}

View the updated dataframe
print(strawberries)

```